Evidence 3 - Parallelizing a FFN-Layer

Moritz Trieu

May 2026

1 Introduction

The problem chosen was the calculation of a Feedforward Network (FFN) layer, one of the types of layers used in Deep Neural Networks for machine learning. In it's most basic form, a FFN layer can be written mathematically as $\mu(Ax + b)$, where $A \in N \times N$ and $b \in N$ are the attributes of the layer and $x \in N$ the input vector. The non-linear function μ is called the activation function and is ignored for the purposes of this evidence. In application N can be very large for example $N > 10,000$. In this case calculating the matrix-vector product and the vector-vector sum can very demanding for a single core or thread. To speed up the process, the paradigm of parallization was chosen. This splits up the operations onto different cores that can calculate different elements of the solution at the same time. In an optimal world this would reduce the computation time by the number of available cores $T_{\text{parallel}} = \frac{T_{\text{single}}}{N_{\text{cores}}}$. This is especially helpful if a large number of cores is available such as the usage GPUs in machine learning.

2 Model

Splitting matrix (or vector) addition into multiple parts is straight forward, each thread can attach to one unique cell in the solution matrix, add the corresponding cells of the input matrices and write the solution into memory. Therefore, each cell can be extracted into an individual task. Thus the main challenge is the index linearization from the host matrix to the device vector.

Matrix-vector multiplication on the other hand becomes a bit more complicated. Calculating the i_{th} component of the output vector is defined as $c_i = \sum_{j=0}^{N-1} a_{ij} \cdot x_i$. Due to the sum, the easiest way to split up the task is for each element of the output vector. This limits the number of tasks that can be generated and the number of operations per thread/ task grows with the problem size.

2.1 Linearization of Indices

When copying matrices to the GPU memory, they get linearized row by row into an array as can be seen in Fig. 1. The mapping from matrix to vector can

$$\begin{bmatrix} a_{11} & \dots & a_{1N} \\ a_{21} & \dots & a_{2N} \\ \vdots & \ddots & \vdots \\ a_{N1} & \dots & a_{NN} \end{bmatrix} \rightarrow \begin{bmatrix} a_{11} & \dots & a_{1N} & a_{21} & \dots & a_{2N} & \dots & a_{N1} & \dots & a_{NN} \end{bmatrix}$$

Figure 1: Linearizing a matrix by appending each row onto the previous.

be calculated by $i_{vec} = i_{mat} \cdot N + j_{mat}$. From a visual standpoint this means column index gets offset by the number of full rows prior.

3 Implementation

4 Test

5 Analysis